Forecast-Grounded Decision Question Answering with Verifiable Reasoning

for Natural Gas Pipeline Transient Operation

Project Proposal / Task Statement
(Translated from Chinese)

June 2026

Item	Description
Research foundation	PipeFormer already exists and handles transient forecasting for natural gas pipeline networks. PipeClaw / Pipeline-Agent already provides pipeline question answering, tool orchestration, and a verifiable computational reasoning framework.
Positioning of this work	This work does not rebuild Pipeline-Agent, nor does it retrain PipeFormer. Instead, it constructs teacher traces grounded in forecast outputs and pipeline engineering constraints, and distills an external lightweight decision model from them.
Core objective	Enable a lightweight model to reliably perform, within PipeClaw, condition parsing, PipeFormer invocation, constraint checking, evidence extraction, and dispatch-recommendation generation.
Target journal orientation	Emphasis on natural gas pipeline network transient operation, operational risk identification, pressure / flow / linepack / compressor constraints, human-intervention judgment, and auditable dispatch recommendations.

Table of Contents

Overall Design

This document is organized around “Forecast-Grounded Decision Question Answering with Verifiable Reasoning for Natural Gas Pipeline Transient Operation,” and splits the research content into three interconnected tasks.

The research foundation of this work is the existing PipeFormer and PipeClaw. PipeFormer already handles transient forecasting for the gas pipeline network, and PipeClaw already provides network question answering, tool invocation, and a verifiable computational reasoning system framework. Therefore, this work does not rebuild another pipeline network agent system, nor does it frame its contribution as improving PipeFormer's forecast accuracy.

The problem this work actually addresses is as follows: Pipeline-Agent / PipeClaw has already demonstrated that a strong model combined with tool invocation and verifiable computational reasoning can be used for gas pipeline decision support; however, this pipeline typically depends on a fairly strong large model to perform task parsing, tool-call planning, computation-result interpretation, and final answer generation. This work further investigates how to transform this “forecast → verification → evidence → answer” verifiable reasoning process into trainable, reusable, and deployable teacher trace data, and to distill it into an external lightweight decision model, enabling that model to be integrated into the PipeClaw system at low cost, with low hallucination, and in a stable manner.

Accordingly, the core innovation of this work does not lie in “building another Pipeline-Agent,” but in the following four aspects:

- Explicitly incorporating gas pipeline network operating rules and engineering constraints into the teacher-answer generation process.
- Constructing a pipeline-network-operation teacher trace dataset based on PipeFormer forecast outputs and constraint-verification results.
- Distilling an external lightweight decision model so that it learns condition parsing, tool invocation, constraint judgment, evidence extraction, and dispatch-answer generation.
- Integrating the distilled model into PipeClaw and validating its engineering reliability, cost advantage, and auditability in gas pipeline transient-operation question answering.

This work is divided into three tasks:

Task	Name	Core Function
Task 1	Teacher Trace dataset generation based on existing PipeFormer and pipeline engineering constraints	Converts raw scenario questions into verifiable supervision data containing forecast results, constraint verification, evidence variables, and standard answers.
Task 2	Distillation and fine-tuning of an external lightweight decision model for pipeline constraints	Trains a student model to learn condition parsing, tool invocation, constraint judgment, evidence extraction, and dispatch-answer generation.
Task 3	System evaluation and pipeline engineering value validation of the distilled model integrated into PipeClaw	Integrates the distilled model into PipeClaw and validates its effectiveness from both engineering-metric and system-metric perspectives.

The overall technical chain is as follows:

Original PipeClaw / PipeFormer scenario question → condition parsing → PipeFormer transient forecast → pipeline engineering constraint verification → structured evidence extraction → teacher answer generation → teacher trace dataset construction → lightweight student model distillation → student model integration into PipeClaw → joint evaluation of engineering risk judgment, dispatch recommendations, and system cost.

This work distinguishes three categories of objects in particular:

Object	Positioning	Is it modified in this work?
PipeFormer	Existing gas pipeline network transient forecasting model, responsible for forecasting future pressure, flow, linepack, compressor load, and other operating states.	Not the primary object of modification; used as a forecasting tool.
PipeClaw	Existing system framework and tool-orchestration platform, responsible for receiving questions, invoking tools, saving traces, and returning final answers.	Used as the integration platform; not the model being distilled.
Student model	The external lightweight decision model distilled and fine-tuned in this work, responsible for question understanding, tool invocation, evidence extraction, and answer generation.	The core object of modification in this work.

Distinction from Pipeline-Agent

The main contribution of Pipeline-Agent is to build a comprehensive decision-support system for natural gas pipelines, achieving a closed loop from user question to tool invocation, script execution, computational verification, and final answer through verifiable computational reasoning.

This work does not duplicate that systemic contribution. The incremental contribution of this work is:

Pipeline-Agent / PipeClaw Focus	This Work's Focus
Building a comprehensive decision-support system for natural gas pipelines.	Transforming the verifiable reasoning process into trainable teacher traces.
Demonstrating that strong model + tools + verifiable computation can be used for pipeline operation decision support.	Investigating how to distill the forecast → verification → explanation pipeline into a lightweight model.
Focus is on system-level agent runtime and verifiable computational reasoning.	Focus is on constructing supervised data from PipeFormer forecast results, pipeline engineering rules, constraint verification, and evidence.
Addresses whether the system can reliably perform pipeline decision support.	Addresses whether a lightweight model can learn this verifiable process while maintaining engineering reliability within PipeClaw.
Contribution is primarily at the system-framework level.	Contribution is primarily constraint-aware teacher-trace construction, trace-level distillation, and engineering evaluation.

Unified Glossary

Term	Meaning
PipeFormer	Existing gas pipeline network transient forecasting model, used to forecast future pressure, flow, linepack, compressor load, and other states given the current operating condition, boundary disturbance, and forecast horizon.
PipeClaw	Existing intelligent question-answering and tool-orchestration system for gas pipeline networks, used to host models, invoke tools, save traces, and return user answers.
Pipeline-Agent	The comprehensive gas pipeline decision-support system from the existing paper, whose core is completing pipeline operation problem analysis through verifiable computational reasoning.
Teacher model	The strong model used to generate high-quality standard answers and tool-call trajectories. It does not answer out of thin air, but organizes answers based on PipeFormer outputs, engineering constraint verification, and structured evidence.
Student model	The external lightweight decision model to be distilled and fine-tuned in this work. It is ultimately integrated into PipeClaw, taking on question parsing, tool invocation, evidence extraction, risk judgment, and answer generation tasks.
Teacher answer	A standard answer supported jointly by PipeFormer forecast results, rule-verification results, and structured evidence.
Teacher trace	The complete supervision trajectory including task parsing, tool invocation, forecast results, constraint verification, evidence extraction, and final answer.
Forecast-grounded decision QA	Forecast-driven decision question answering, referring to a system that first invokes a forecasting model to obtain future operating states, then completes risk judgment and dispatch recommendations based on the forecast results.
Verifiable reasoning	Reasoning in which key judgments in the answer must be traceable to forecast results, engineering thresholds, rule verification, or structured evidence.
Constraint-aware distillation	Distillation in which the student model learns not only the final answer, but also pipeline constraint judgment, risk labels, tool invocation, and the evidence-extraction process.

Task 1: Teacher Trace Dataset Generation Based on Existing PipeFormer and Pipeline Engineering Constraints

1.1 Research Objective

The objective of Task 1 is to generate a teacher-answer and teacher-trace dataset for gas pipeline network operation question answering, based on the existing PipeFormer forecasting model and the network operation scenario questions already present in PipeClaw.

The existing JSON files mainly contain task-input information such as user_input, scenario_description, scenario_type, and sessions, but lack standard answers, forecast results, tool-call trajectories, engineering constraint-verification results, and evidence. Therefore, they cannot be used directly for supervised fine-tuning.

Task 1 needs to transform these raw questions into complete, verifiable supervision samples. Each sample must contain not only the final answer, but also the user question, structured task parsing, PipeFormer tool invocation, PipeFormer forecast summary, pipeline engineering constraint-verification results, key evidence variables, risk level, whether human intervention is required, dispatch recommendations, and the final teacher answer.

The core of Task 1 is not to have a large model directly generate answers, but to establish a verifiable data-generation pipeline of "user question → PipeFormer forecast → pipeline rule verification → evidence extraction → teacher answer."

1.2 Distinction from Pipeline-Agent

Pipeline-Agent has already established a comprehensive decision-support system for gas pipelines and improved answer reliability through verifiable computational reasoning. Task 1 does not rebuild an agent system again; instead, it converts the verifiable reasoning pipeline within Pipeline-Agent / PipeClaw into data, supervision, and distillable form.

In other words, the focus of Task 1 is not "enabling the system to compute," but "organizing the process that the system computes, verifies, and explains into teacher traces." This allows the subsequent student model to learn not just the final natural-language answer, but the complete forecast → verification → evidence → answer logic.

1.3 Data Inputs

The core inputs come from JSON files already existing in PipeClaw, including question_all.json, question_template_80.json, question_template_all.json, pipeclaw_dataset_v2.json, Pipeline_Full_Life_Cycle_Test_Dataset-v4.json, and Pipeline_Full_Life_Cycle_Test_Dataset-v7.json.

Among these, the question-series files are mainly used to expand general network question-answering templates; pipeclaw_dataset_v2 can be used for multi-turn tasks and comprehensive operation scenarios; and Pipeline_Full_Life_Cycle_Test_Dataset-v4 and v7 are more suitable as seeds for PipeFormer-in-the-loop forecast, verification, and dispatch question-answering tasks.

Each computable task needs to be organized into an input acceptable to PipeFormer, including case_id, current operating-condition number, boundary conditions, disturbance variable, disturbance direction, disturbance magnitude, forecast horizon, nodes/segments/equipment requiring attention, state variables to output, and constraint-verification types to execute.

1.4 Pipeline Engineering Constraint Inputs

The most important modification in Task 1 is to explicitly introduce pipeline engineering rules into the teacher-trace generation process. A library of gas pipeline network operating constraints needs to be established.

Constraint category	Main content
Pressure constraints	Node pressure lower bound, node pressure upper bound, key-node pressure margin, out-of-limit duration.
Flow constraints	Segment flow change, maximum transmission capacity, boundary flow change rate, supply-demand balance.
Linepack constraints	Linepack decline magnitude, linepack recovery capability, short-term peak-shaving capacity, warning threshold.
Compressor constraints	Compressor load, compression ratio, rotational speed, power, operating envelope, and regulation margin.
Equipment and regulation constraints	Valve opening, pressure-regulating equipment range, allowable adjustment magnitude of boundary control variables.
Abnormality warning rules	Rules for reviewing abnormal pressure drops, sudden flow changes, and potential leaks or equipment anomalies.
Human-intervention rules	no_intervention, monitoring_only, operator_attention_required, immediate_intervention_required.
Dispatch priority rules	Safety first, then supply assurance, then equipment protection, and finally energy consumption and cost.

1.5 Pipeline Rule System Design

1.5.1 Pressure Safety Rules

Pressure is the core variable for gas pipeline network operating safety. The teacher trace must record the minimum pressure, maximum pressure, pressure-violation nodes, and pressure-violation duration. If any key node's pressure falls below the lower bound, it is flagged as pressure_violation; if a node's pressure is close to but has not crossed the lower bound, it is flagged as pressure_warning; if multiple end-user nodes simultaneously approach the pressure lower bound, the risk level is raised; if the pressure-recovery time is too long, human attention or dispatch intervention is recommended.

1.5.2 Flow and Supply-Demand Balance Rules

Flow changes reflect boundary disturbances, changes in user load, and changes in segment transmission capacity. The teacher trace needs to record flow-change magnitude, abnormal-flow segments, and supply-demand balance status. If a segment's flow exceeds the allowed transmission capacity, it is flagged as flow_capacity_violation; if flow changes beyond the set threshold within a short time, it is flagged as flow_ramp_warning; if the supply-demand gap continues to widen, it is flagged as balance_warning.

1.5.3 Linepack Rules

Linepack reflects short-term peak-shaving capacity and system buffering capacity. The teacher trace needs to record the linepack change rate, the time of minimum linepack, and the linepack warning status. If linepack decline exceeds the warning threshold, it is flagged as linepack_warning; if linepack continues to decline while pressure also approaches its lower bound, the risk level is raised; if linepack recovery is insufficient, increasing upstream injection or adjusting the compressor strategy is recommended.

1.5.4 Compressor Operation Rules

The compressor is a key piece of equipment in gas pipeline network dispatch. The teacher trace needs to record compressor load, compression ratio, power, and whether it is approaching the operating envelope. If compressor load exceeds the upper limit, it is flagged as compressor_overload; if the compression ratio approaches the boundary, it is flagged as compressor_ratio_warning; if the compressor still has regulation margin, the dispatch recommendation may prioritize compressor adjustment.

1.5.5 Human-Intervention and Dispatch-Priority Rules

The teacher answer cannot merely state “safe” or “unsafe”—it must also judge whether human intervention is required. Human-intervention labels can be divided into no_intervention, monitoring_only, operator_attention_required, and immediate_intervention_required. Dispatch recommendations should follow the safety-first principle: the first priority is eliminating pressure violations and end-user supply-assurance risk; the second priority is avoiding compressor overload and equipment boundary violations; the third priority is restoring the linepack safety margin; the fourth priority is reducing flow fluctuations; and the fifth priority is optimizing energy consumption and operating cost after safety constraints are satisfied.

1.6 Teacher Trace Generation Process

Step	Description
User-question parsing	The teacher model or a rule-based parser converts user_input into a structured task, identifying case_id, disturbance variable, disturbance magnitude, forecast horizon, and required constraint checks.
PipeFormer forecast	Based on the parsed_task, invokes the existing PipeFormer, outputting future time-series results of pressure, flow, linepack, and compressor load, and extracting a forecast summary.

Step	Description
Rule verification	Based on PipeFormer's outputs, executes the constraint checker to determine whether pressure, flow, linepack, compressor, and supply-demand-balance constraints trigger a warning or violation.
Evidence extraction	From the forecast summary and rule-verification results, extracts the key evidence variables supporting the final answer, including numerical values, thresholds, margins, states, components, and timing.
Teacher-answer generation	The teacher model generates the final answer based on user_input, prediction_summary, constraint_check, and evidence, and must not fabricate data that was not provided.

1.7 Teacher Trace Data Format

Field	Meaning
sample_id	Sample number
scenario_id	Scenario number
scenario_type	Scenario type
user_input	Original user question
parsed_task	Structured task-parsing result
tool_calls	Tool-invocation record
tool_outputs	Tool output results
prediction_summary	PipeFormer forecast summary
constraint_check	Engineering constraint-verification result
evidence	Key evidence variables
risk_level	Risk level
manual_intervention_label	Human-intervention label
dispatch_recommendation	Dispatch recommendation
final_answer	Final teacher answer
quality_flag	Quality flag

1.8 Quality Control

- Schema check: verifies whether each sample contains parsed_task, tool_calls, prediction_summary, constraint_check, evidence, and final_answer.
- Numerical consistency check: verifies whether the values referenced in final_answer come from prediction_summary or evidence. If a nonexistent pressure, flow, node, segment, or compressor metric appears, it should be flagged as hallucination.
- Rule consistency check: verifies whether the risk judgment in final_answer is consistent with constraint_check. For example, when pressure_violation = true, final_answer must not judge the situation as entirely safe.
- Dispatch-recommendation consistency check: verifies whether the dispatch recommendation follows the safety-first principle. When pressure has already fallen below the lower bound, the recommendation must not prioritize reducing upstream injection; when the compressor is already overloaded, the recommendation must not continue to suggest raising compressor load.
- Manual spot-checking: it is recommended that 20%–30% of samples undergo manual spot-checking, focusing on whether condition parsing, tool invocation, risk level, and dispatch recommendations are reasonable.

1.9 Task 1 Output Deliverables

Output type	Files / content
Teacher trace dataset	teacher_trace_train.jsonl, teacher_trace_valid.jsonl, teacher_trace_test.jsonl, teacher_trace_schema.json, teacher_trace_quality_report.xlsx
Pipeline constraint rule library	pipeline_constraints.json, pressure_rules.json, flow_rules.json, linepack_rules.json, compressor_rules.json, intervention_rules.json, dispatch_priority_rules.json
Data statistics tables	Total sample count, task-type distribution, constraint-type distribution, risk-level distribution, human-intervention label distribution, average number of evidence items, quality-check pass rate, etc.

1.10 Core Statement of Task 1 Results

- The existing JSON files are task seeds, not directly usable supervised fine-tuning data.
- Based on the existing PipeFormer and pipeline engineering constraints, task seeds can be transformed into verifiable teacher traces.
- The teacher answer is not freely generated by a large model, but is supported by forecast results, constraint verification, and evidence.
- Pipeline engineering rules are explicitly incorporated into the data-generation pipeline, so that the subsequent student model learns gas pipeline network operational decision logic rather than a generic question-answering style.

1.11 Items Requiring Further Investigation and Refinement

Follow-up work on Task 1 needs to focus on investigating PipeFormer's actual input/output interface, forecast horizon settings, supported variable types, and batch-invocation stability. It also needs to further investigate the sources of gas pipeline network operating-safety-constraint thresholds, including pressure upper/lower bounds, compressor operating boundaries, linepack warning standards, boundary-flow adjustment ranges, and human-intervention rules. If real engineering thresholds are not yet available, a parameterized rule library can be established first, with a sensitivity analysis performed in the paper.

Task 2: Distillation and Fine-Tuning of an External Lightweight Decision Model for Pipeline Constraints

2.1 Research Objective

The objective of Task 2 is to distill and fine-tune an external lightweight decision model based on the teacher trace dataset generated in Task 1, so that it learns the key capabilities involved in forecast-grounded decision question answering for gas pipeline networks. This work does not distill PipeFormer, nor does it distill PipeClaw; rather, it distills an external student model so that it can serve as the reasoning / QA / decision module within PipeClaw.

This student model needs to learn how to understand gas pipeline network operation questions, parse operating conditions and disturbance conditions, plan PipeFormer and rule-checker invocations, read PipeFormer forecast results, judge risk based on pipeline constraints, extract key evidence, and generate dispatch recommendations and final answers.

2.2 Distinction from Pipeline-Agent

Pipeline-Agent mainly relies on a strong model for planning, computation invocation, and interpretation. The focus of Task 2 is to compress this strong-model-driven verifiable reasoning pipeline into a lightweight model. It needs to learn engineering judgments such as pressure violations, linepack warnings, and dispatch recommendations, rather than merely learning a natural-language answering style.

2.3 Training Task Design

Training task	Objective
Condition-parsing training	Input user_input, output parsed_task; learn to identify case_id, disturbance variable, disturbance magnitude, forecast horizon, and required checks.
Tool-call planning training	Input parsed_task, output tool_calls; learn when to invoke PipeFormer, the constraint_checker, and the evidence_extractor.
Pipeline constraint judgment training	Input prediction_summary and constraint_check, output risk_level, constraint_status, and manual_intervention_label.
Evidence-extraction training	Input the forecast summary and constraint verification, output evidence; learn to extract key evidence for pressure, flow, linepack, and compressor.
Final answer generation training	Input user_input, prediction_summary, constraint_check, and evidence, output final_answer.

2.4 Distillation Strategy

Strategy	Description
Answer-only SFT	Trains the model using only user_input and final_answer. Can serve as a baseline, but tends to learn only the answering style.
Trace-level SFT	Trains the model using the complete teacher trace, including parsed_task, tool_calls, prediction_summary, constraint_check, evidence, risk_label, and final_answer. The primary method of this work.
Constraint-aware multi-task SFT	Splits the training task into five subtasks: condition parsing, tool invocation, constraint judgment, evidence extraction, and answer generation.

Strategy	Description
Preference optimization	After SFT, constructs preference samples that favor answers that correctly cite evidence, follow constraint verification, prioritize safety, and acknowledge uncertainty.

2.5 Model Selection

The student model can be chosen from small-to-medium-scale open-source models, such as Qwen, DeepSeek, or Llama. Selection criteria include Chinese and English comprehension ability, structured JSON output capability, tool-call learning capability, long-context processing capability, local deployment cost, inference speed, maturity of the LoRA / QLoRA fine-tuning ecosystem, and the difficulty of integration with PipeClaw.

If the goal is to reduce the cost of strong-model invocation within PipeClaw, it is recommended to first test models at the 7B or 14B scale. Larger models can serve as the teacher or as an upper-bound baseline.

2.6 Training Set Splitting

- Random split: randomly divides the training, validation, and test sets, used for baseline performance evaluation.
- Case holdout: reserves some mock_test cases entirely as a test set, used to examine the model's generalization to new operating conditions.
- Scenario type holdout: reserves some scenario classes as a test set, used to examine the model's generalization to new task types.
- Constraint type holdout: reserves some constraint types as a test set—for example, if compressor overload rarely appears in training, the test set can specifically evaluate whether the model can handle compressor constraints.
- Intervention variable holdout: reserves some disturbance variables as a test set, used to examine the model's generalization to new boundary variables, equipment variables, or control variables.

2.7 Task 2 Output Deliverables

Output type	Content
Fine-tuned model files	Student model, LoRA adapter, tokenizer config, training config, inference script, model card.
Training logs	Training loss, validation loss, learning rate, epoch, batch size, sequence length, GPU memory usage, training time.
Model evaluation results	Task-parsing accuracy, tool-call success rate, constraint-judgment accuracy, evidence consistency, risk-classification accuracy, manual-intervention accuracy, dispatch-recommendation feasibility, hallucination rate, answer completeness, JSON validity rate.

2.8 Core Statement of Task 2 Results

- Answer-only SFT can only improve answering style; it cannot adequately guarantee correct tool invocation and a correct engineering evidence chain.
- Trace-level distillation enables the student model to more stably learn condition parsing, tool invocation, constraint judgment, and evidence extraction.
- Introducing pipeline engineering constraints via constraint-aware distillation reduces numerical hallucination and erroneous dispatch recommendations.

- After distillation, the student model can approach the teacher model's forecast-grounded decision question-answering ability at a lower inference cost.
- This student model is suitable as a lightweight reasoning / QA / decision module within PipeClaw.

2.9 Items Requiring Further Investigation and Refinement

Follow-up work on Task 2 needs to compare the performance differences of different open-source models in structured output, tool invocation, and engineering constraint judgment. It also needs to compare the effectiveness and cost of LoRA, QLoRA, full-parameter fine-tuning, and prompt tuning. If the training data scale is small, sample diversity should be increased through template expansion, disturbance-variable expansion, forecast-horizon expansion, scenario rewriting, and multi-teacher generation. In addition, a hallucination-detection mechanism needs to be established, focusing on checking whether the model fabricates nonexistent pressure, flow, node, segment, or compressor metrics, or dispatch actions.

Task 3: System Evaluation and Pipeline Engineering Value Validation of the Distilled Model Integrated into PipeClaw

3.1 Research Objective

The objective of Task 3 is to integrate the student model obtained in Task 2 into PipeClaw, forming a forecast-grounded, verifiable decision question-answering system for gas pipeline network transient operation, and to evaluate it from both pipeline-engineering-metric and system-metric perspectives.

The questions Task 3 needs to answer include: Can the distilled student model stably parse user questions within PipeClaw? Can it correctly invoke the existing PipeFormer model? Can it correctly read PipeFormer's outputs? Can it judge risk based on pressure, flow, linepack, and compressor rules? Can it generate traceable, low-hallucination, engineering-grade dispatch recommendations? And compared with a strong model, a prompt-only method, and answer-only SFT, does it reduce cost, improve speed, and maintain engineering judgment accuracy?

3.2 System Integration Plan

After integration into PipeClaw, the student model handles user-question understanding, structured task generation, tool-call planning, PipeFormer-output interpretation, constraint-verification-result interpretation, key-evidence extraction, risk-level judgment, final-answer generation, and multi-turn context retention. PipeClaw retains the front-end and back-end interfaces, tool registration, PipeFormer invocation, rule-checker invocation, trace saving, result display, and user-interaction management.

The system execution flow is as follows: user inputs a network-operation question → student model parses it into a structured task → PipeClaw invokes PipeFormer → PipeFormer returns future-state forecast results → PipeClaw invokes the constraint checker → student model reads the forecast and verification results → student model extracts evidence and judges risk → student model generates the final answer → PipeClaw saves the complete trace and returns it to the user.

3.3 Pipeline Operation Scenario Matrix

Scenario	Pipeline problem	Evaluation focus
End-user load surge scenario	After a user's load increases, does end-of-line pressure fall below the lower bound, requiring dispatch intervention?	Minimum node pressure, pressure margin, out-of-limit duration, end-user supply-assurance risk, human-intervention judgment.
Upstream supply reduction scenario	After upstream injection decreases, does linepack continue to decline, and can the system maintain supply?	Linepack change, supply-demand gap, pressure-recovery capability, peak-shaving capacity, intervention recommendation.
Compressor setpoint adjustment scenario	Does raising compressor outlet pressure or speed improve downstream pressure, and does it cause compressor overload?	Compressor load, compression-ratio boundary, pressure-recovery effect, equipment risk, dispatch priority.
Boundary flow reallocation scenario	After adjusting flow at different boundaries, can local pressure or flow constraints be relieved?	Constraint-violation-reduction effect, flow-reallocation effect, degree of pressure improvement, linepack impact, plan ranking.

Scenario	Pipeline problem	Evaluation focus
Segment bottleneck / throttling scenario	Does local throttling or a bottleneck cause upstream pressure buildup or downstream supply shortfall?	Key segment flow, upstream/downstream pressure change, flow anomaly, downstream supply-assurance risk, dispatch recommendation.
Abnormal pressure-drop warning scenario	Does a local abnormal pressure drop require triggering human review or a safety warning?	Abnormal node identification, pressure-drop magnitude, anomaly duration, potential leak or equipment-anomaly indication, human-review recommendation.

3.4 Comparison Methods

Method	Description
LLM-only	Does not invoke PipeFormer or the rule checker; answers the user's question directly. Used to demonstrate that a pure language model is prone to numerical hallucination and unverifiable judgments.
Prompt-only with tools	Uses a general-purpose model, guided via prompting to invoke PipeFormer and the checker, but without fine-tuning.
Answer-only SFT	Trains the model using only final_answer. Used to demonstrate that learning only the final answer is inferior to learning the complete trace.
Trace-distilled student model	Trains the model using the complete teacher trace.
Constraint-aware trace-distilled model	Building on trace-level distillation, explicitly adds pipeline constraints, risk labels, human-intervention labels, and dispatch-priority rules. The complete method of this work.
Teacher model upper bound	Uses the strong teacher model as an upper-bound comparison, evaluating the gap between the student model and the teacher model.

3.5 Evaluation Metrics

Metric category	Metrics
Pipeline engineering metrics	Pressure-violation judgment accuracy, flow-anomaly judgment accuracy, linepack-warning judgment accuracy, compressor-overload judgment accuracy, supply-demand-imbalance judgment accuracy, end-user supply-assurance-risk judgment accuracy, human-intervention judgment accuracy, risk-level classification accuracy, key-risk-node identification accuracy, key-risk-time identification accuracy, dispatch-recommendation feasibility, dispatch-recommendation priority consistency.
QA system metrics	Task-parsing accuracy, PipeFormer call success rate, constraint-checker call success rate, tool-parameter accuracy, structured-output validity rate, evidence hit rate, numerical-reference consistency, hallucination rate, final-answer completeness, average response latency, inference cost.
Verifiability metrics	Whether key judgments in the answer are traceable to evidence; whether values originate from PipeFormer or checker outputs; whether fabricated nodes, segments, or equipment appear; whether judgments contradict constraint_check; whether uncertainty is correctly stated; whether human review is recommended in high-risk scenarios; whether the safety-first dispatch principle is followed.

3.6 Experimental Design

- Single-turn forecast QA experiment: input a single-turn network-operation question, testing whether the system can complete condition parsing, PipeFormer invocation, constraint verification, evidence extraction, risk judgment, and final-answer generation.
- Multi-turn dispatch QA experiment: simulates consecutive user follow-up questions—e.g., changes in disturbance magnitude, changes in forecast horizon, comparison of compressor vs. boundary-flow options—evaluating context retention, multi-option comparison, and dispatch-explanation capability.
- Generalization experiments: sets up case holdout, scenario type holdout, constraint type holdout, intervention variable holdout, and forecast horizon holdout.

- Tool-failure and uncertainty experiments: simulates PipeFormer call failure, missing forecast results, missing constraint thresholds, or incomplete user-question information, evaluating whether the model can correctly state uncertainty or recommend human review.
- Ablation experiments: compares the complete model, the model without evidence, without constraint check, without tool call, without dispatch priority rule, trained on final answer only, and the prompt-only model.

3.7 Output Deliverables

Output table	Fields / content
Sample-level evaluation table	sample_id, task_type, method, parse_correct, tool_call_correct, constraint_correct, evidence_correct, answer_correct, dispatch_feasible, hallucination_flag, latency, cost.
Pipeline engineering metric summary table	pressure_violation_accuracy, flow_abnormality_accuracy, linepack_warning_accuracy, compressor_overload_accuracy, supply_demand_balance_accuracy, manual_intervention_accuracy, risk_classification_accuracy, dispatch_feasibility.
System metric summary table	task_parsing_accuracy, tool_call_success_rate, json_validity_rate, evidence_consistency, hallucination_rate, answer_completeness, average_latency, relative_cost.
Ablation experiment table	without_trace, without_evidence, without_constraint_check, without_dispatch_priority, answer_only, full_model.

3.8 Figures and Charts

- System framework diagram: shows the relationships among user input, the student model, PipeClaw, PipeFormer, the constraint checker, and the final answer.
- Teacher trace construction flow diagram: shows how a raw question is converted into a parsed task, tool call, prediction summary, constraint check, evidence, and final answer.
- Pipeline constraint rule library schematic: shows how pressure, flow, linepack, compressor, and human-intervention rules participate in verification.
- Comparison charts of different methods' tool-call success rate, risk-judgment accuracy, hallucination rate, response latency, and inference cost.
- Typical network-disturbance case diagram: shows, for a given disturbance, how the system forecasts future states, identifies risk nodes, extracts evidence, and provides dispatch recommendations.
- Ablation experiment chart: shows the performance degradation after removing evidence, constraint check, or the dispatch priority rule.

3.9 Core Statement of Task 3 Results

- The LLM-only method is prone to numerical hallucination and unverifiable judgments in network-operation QA and is not suitable as an independent basis for engineering decisions.
- Prompt-only with tools can invoke PipeFormer, but tool-call format, evidence extraction, and constraint-judgment stability are insufficient.
- Answer-only SFT can only improve answering style; it cannot adequately guarantee correct tool invocation and a correct engineering evidence chain.

- The trace-distilled student model more stably completes condition parsing, PipeFormer invocation, constraint verification, evidence extraction, and dispatch-recommendation generation.
- Constraint-aware trace distillation further improves the accuracy of pressure-violation, linepack-warning, compressor-overload, and human-intervention judgments.
- Integrating the student model into PipeClaw can reduce the cost of invoking the strong teacher model, improve response speed, and maintain high engineering-judgment accuracy.
- The pipeline engineering value of this method lies in converting PipeFormer's numerical forecast results into risk judgments and dispatch recommendations that are understandable, traceable, and auditable by operating personnel.

3.10 Items Requiring Further Investigation and Refinement

Follow-up work on Task 3 needs to focus on investigating PipeClaw's actual model-integration approach, tool-call protocol, trace-recording format, and front-end display method. It needs to confirm whether the student model is integrated as an independent API service or replaces PipeClaw's existing LLM-invocation module.

It also needs to distinguish output permissions under different risk levels. For low-risk questions, the model can directly provide explanatory recommendations; for questions involving pressure violations, compressor overload, insufficient linepack, and human intervention, the system must mandatorily go through PipeFormer forecasting and rule verification; for high-risk or highly uncertain questions, the system should recommend human review rather than directly issuing a forceful dispatch instruction.

If real SCADA data, network simulator outputs, or expert-annotated cases can be obtained subsequently, the system can be further expanded from mock cases to real operating scenarios, improving the paper's engineering credibility.

Final Paper Positioning Recommendations

Suggested title: Forecast-Grounded Decision Question Answering with Verifiable Reasoning for Natural Gas Pipeline Transient Operation.

Suggested research thread: This work does not retrain PipeFormer, nor does it duplicate the construction of the Pipeline-Agent / PipeClaw system. Instead, it proposes a verifiable decision question-answering distillation method based on the existing PipeFormer forecasting capability and pipeline engineering constraints. This method first uses PipeFormer to generate transient forecast results for the gas pipeline network, and performs engineering verification through pressure, flow, linepack, compressor, and human-intervention rules; it then organizes the forecast results, constraint verification, structured evidence, and final answer into a teacher trace dataset; and finally distills an external lightweight decision model based on this dataset and integrates it into PipeClaw, achieving low-cost, traceable, low-hallucination gas pipeline network operation question answering and dispatch-recommendation generation.

It is recommended to distill the contribution into three points:

1. Constructing a teacher-answer / teacher-trace generation pipeline based on existing PipeFormer forecast results and pipeline engineering constraint verification.
2. Proposing a constraint-aware, trace-level distillation method for gas pipeline network operation question answering, enabling an external lightweight model to learn condition parsing, tool invocation, constraint judgment, evidence extraction, and dispatch-answer generation.
3. Integrating the distilled decision model into PipeClaw and validating its effectiveness in terms of pressure-violation judgment, linepack warning, compressor overload, human-intervention judgment, dispatch-recommendation feasibility, tool-call stability, hallucination rate, response speed, and cost.

Phrasings to Avoid

- Do not write “this work improves PipeFormer's forecast accuracy” unless PipeFormer is actually retrained and evaluated.
- Do not write “this work distills PipeClaw,” because PipeClaw is a system framework, not the model being distilled.
- Do not frame the paper's main thread as an “LLM fine-tuning method,” or it will skew toward computer science and be insufficiently aligned with a natural-gas-pipeline engineering journal.
- Do not duplicate Pipeline-Agent's systemic contribution; instead, emphasize that this work is a lightweight, constraint-aware, distillation-based extension of its verifiable computational reasoning pipeline.

Suggested framing: This work combines the existing PipeFormer's transient forecasting capability, gas pipeline network engineering constraint verification, and lightweight decision-model distillation to build a forecast-grounded, verifiable decision question-answering system for gas pipeline network transient operation. Unlike the existing Pipeline-Agent, which focuses on the system framework and verifiable computational reasoning, this work focuses on how to transform the engineering decision process of forecast → verification → evidence → answer into teacher traces, and to distill it into a deployable lightweight model. Its core value lies in converting complex numerical forecast results into risk judgments and dispatch recommendations that are understandable, traceable, and auditable by operating personnel, while reducing the cost of invoking a strong model and the risk of answer hallucination.

References

- Pipeline-Agent: A Comprehensive Decision Support System for Natural Gas Pipelines via Verifiable Computational Reasoning.
- PipeClaw / Pipeline-Agent GitHub repository and related public dataset files.
- PipeFormer repository and model documentation.